MITRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE, ALWAR

Name of faculty: Dazy Arya

Subject & Subject Code: CF&P,2FY2-08

Semester: II **Session: 2021-22 (Even Sem.)**

Branch: AI&DS,ME,CE, EE **Batch: C,D**

UNIT: II **Date of Submission: 28/03/2023**

Lecture No.	Topics Covered	Total Page
1	'C' Character set,Literals,keywords,identifier,	3
2	data type in C,ASCII Code,	4
3	Variable declaration ,Expression,labels,statements,.	2
4	type of operators,,operator Expression Assocaitivity,	2
5	formatted input output statement,	10
6	precedence of operator,expression evaluation,Data type Conversion,mixed mode arithmetic's	10

References:

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

28/3/2023

Faculty **HOD** **Dean Academics**

LECTURE -01 (UNIT-02)

CHARACTER SET IN C

Characters are used in forming either words or numbers or even expressions in C programming. Characters in C are classified into 4 groups:

Letters

In C programming, we can use both uppercase and lowercase letters of English language

- Uppercase Letters: A to Z
- Lowercase Letters: a to z

Digits

We can use decimal digits from 0 to 9.

Special Characters

C Programming allows programmer to use following special characters:

comma ,	slash /
period .	backslash
semicolon ;	percentage %
colon :	left and right brackets [ ]
question mark ?	left and right parenthesis ( )

White Spaces

In C Programming, white spaces contain:

- Blank Spaces
- Tab
- Carriage Return
- New Line

Letters	A B C D E F G H I J K L M N O P Q R S T U V W X Y Z a b c d e f g h i j k l m n o p q r s t u v w x y z
Digits	0 1 2 3 4 5 6 7 8 9
Special Characters	` ~ ! @ # % ^ & * ( ) _ - + = | { } [ ] : ; ' " / ? < > . ,
White Spaces	Blank space, tab, carriage return, new line

Keywords

Keywords are special words in C programming which have their own predefined meaning. The functions and meanings of these words cannot be altered. Some keywords in C Programming are:

Auto	Break	case	char
Continue	Const	do	default
Double	Else	enum	extern
For	Float	go	If
Int	Long	register	return
Sign	Static	sizeof	short
Struct	Switch	typedef	union
Void	Volatile	while	unsigned

Identifiers

Identifiers are user-defined names of variables, functions and arrays. It comprises of combination of letters and digits. In C Programming, while declaring identifiers, certain rules have to be followed viz.

- It must begin with an alphabet or an underscore and not digits.
- It must contain only alphabets, digits or underscore.
- A keyword cannot be used as an identifier
- Must not contain white space.
- Only first 31 characters are significant.

Let us again consider an example

```
int age1;
float height_in_feet;
```

Here, *age1* is an identifier of integer data type.

Similarly *height_feet* is also an identifier but of floating integer data type,

Note: Every word in C program is either a keyword or an identifier.

.

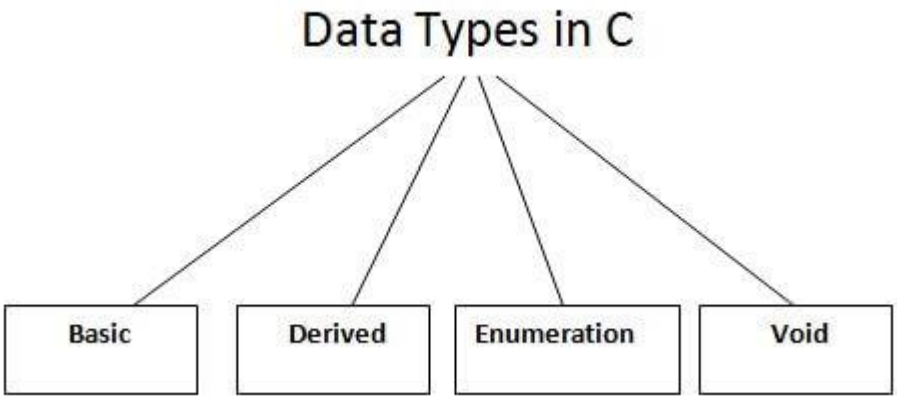
REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -02 (UNIT-02)

DATA TYPES IN C

A data type specifies the type of data that a variable can store such as integer, floating, character, etc.



There are the following data types in C language.

Types	Data Types
Basic Data Type	int, char, float, double
Derived Data Type	array, pointer, structure, union
Enumeration Data Type	Enum
Void Data Type	Void

BASIC DATA TYPES

The basic data types are integer-based and floating-point based. C language supports both signed and unsigned literals.

The memory size of the basic data types may change according to 32 or 64-bit operating system.

Let's see the basic data types. Its size is given according to 32-bit architecture.

Data Types	Memory Size	Range
Char	1 byte	−128 to 127
signed char	1 byte	−128 to 127
unsigned char	1 byte	0 to 255
Short	2 byte	−32,768 to 32,767
signed short	2 byte	−32,768 to 32,767
unsigned short	2 byte	0 to 65,535
Int	2 byte	−32,768 to 32,767
signed int	2 byte	−32,768 to 32,767
unsigned int	2 byte	0 to 65,535
short int	2 byte	−32,768 to 32,767
signed short int	2 byte	−32,768 to 32,767
unsigned short int	2 byte	0 to 65,535
long int	4 byte	-2,147,483,648 to 2,147,483,647
signed long int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned long int	4 byte	0 to 4,294,967,295
Float	4 byte	
Double	8 byte	
long double	10 byte	

What is ASCII code?

The full form of ASCII is the American Standard Code for information interchange. It is a character encoding scheme used for electronics communication. Each character or a special character is represented by some ASCII code, and each ascii code occupies 7 bits in memory.

In C programming language, a character variable does not contain a character value itself rather the ascii value of the character variable. The ascii value represents the character variable in numbers, and each character variable is assigned with some number range from 0 to 127. For example, the ascii value of 'A' is 65.

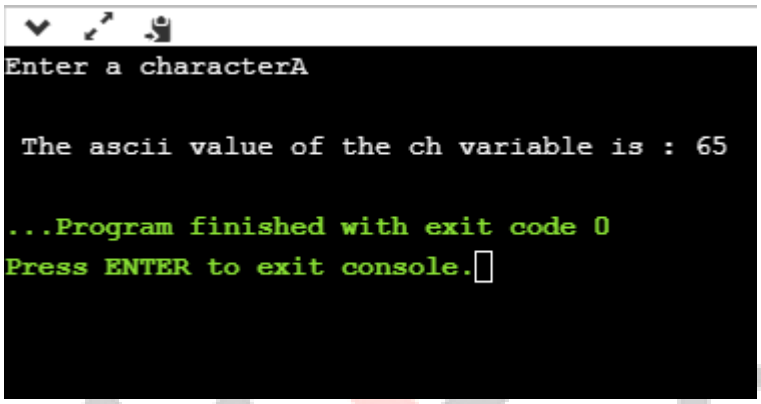
In the above example, we assign 'A' to the character variable whose ascii value is 65, so 65 will be stored in the character variable rather than 'A'.

We will create a program which will display the ascii value of the character variable.

```
1. #include <stdio.h>
2. int main()
3. {
4.     char ch;    // variable declaration
```

```
5.      printf("Enter a character");
6.      scanf("%c",&ch);  // user input
7.      printf("\n The ascii value of the ch variable is : %d", ch);
8.      return 0;
9.  }
```

In the above code, the first user will give the character input, and the input will get stored in the 'ch' variable. If we print the value of the 'ch' variable by using %c format specifier, then it will display 'A' because we have given the character input as 'A', and if we use the %d format specifier then its ascii value will be displayed, i.e., 65.



Output

The above output shows that the user gave the input as 'A', and after giving input, the ascii value of 'A' will get printed, i.e., 65.

Now, we will create a program which will display the ascii value of all the characters.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.   int k;   // variable declaration
5.   for(int k=0;k<=255;k++)  // for loop from 0-255
6.   {
7.       printf("\nThe ascii value of %c is %d", k,k);
8.   }
9.  return 0;
10. }
```

The above program will display the ascii value of all the characters. As we know that ascii value of all the characters starts from 0 and ends at 255, so we iterate the for loop from 0 to 255.

Now we will create the program which will sum the ascii value of a string.

```
1. #include <stdio.h>
2. int main()
3. {
4.     int sum=0;  // variable initialization
5.     char name[20];  // variable initialization
6.     int i=0;  // variable initialization
7.     printf("Enter a name: ");
8.     scanf("%s", name);
9.     while(name[i]!='\0')  // while loop
10.    {
11.        printf("\nThe ascii value of the character %c is %d", name[i],name[i]);
12.        sum=sum+name[i];
13.        i++;
14.    }
15.    printf("\nSum of the ascii value of a string is : %d", sum);
16.    return 0;
17. }
```

In the above code, we are taking user input as a string. After taking user input, we execute the while loop which adds the ascii value of all the characters of a string and stores it in a 'sum' variable.

Output

```
Enter a name: akshita

The ascii value of the character a is 97
The ascii value of the character k is 107
The ascii value of the character s is 115
The ascii value of the character h is 104
The ascii value of the character i is 105
The ascii value of the character t is 116
The ascii value of the character a is 97
Sum of the ascii value of a string is : 741

...Program finished with exit code 0
Press ENTER to exit console.
```

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -03 (UNIT-02)

VARIABLES IN C

A variable is a name of the memory location. It is used to store data. Its value can be changed, and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified.

Let's see the syntax to declare a variable:

type variable_list;

The example of declaring the variable is given below:

- int a;
- float b;
- char c;

Here, a, b, c are variables. The int, float, char are the data types.

We can also provide values while declaring the variables as given below:

```
1. int a=10,b=20;//declaring 2 variable of integer type
2. float f=20.8;
3. char c='A';
```

Rules for defining variables

- A variable can have alphabets, digits, and underscore.
- A variable name can start with the alphabet, and underscore only. It can't start with a digit.
- No whitespace is allowed within the variable name.
- A variable name must not be any reserved word or keyword, e.g. int, float, etc.

Valid variable names:

```
1. int a;
2. int _ab;
3. int a30;
```

Invalid variable names:

```
1. int 2;
2. int a b;
3. int long;
```

Types of Variables in C

There are many types of variables in c:

1. local variable
2. global variable
3. static variable
4. automatic variable
5. external variable

Local Variable

A variable that is declared inside the function or block is called a local variable.

It must be declared at the start of the block.

```
1. void function1(){
2. int x=10;//local variable
3. }
```

You must have to initialize the local variable before it is used.

Global Variable

A variable that is declared outside the function or block is called a global variable. Any function can change the value of the global variable. It is available to all the functions.

It must be declared at the start of the block.

```
1. int value=20;//global variable
2. void function1(){
3. int x=10;//local variable
4. }
```

Static Variable

A variable that is declared with the static keyword is called static variable.

It retains its value between multiple function calls.

```
1. void function1(){
2. int x=10;//local variable
3. static int y=10;//static variable
4. x=x+1;
5. y=y+1;
6. printf("%d,%d",x,y);
7. }
```

If you call this function many times, the local variable will print the same value for each function call, e.g, 11,11,11 and so on. But the static variable will print the incremented value in each function call, e.g. 11, 12, 13 and so on.

Automatic Variable

All variables in C that are declared inside the block, are automatic variables by default. We can explicitly declare an automatic variable using auto keyword.

```
1. void main(){
2. int x=10;//local variable (also automatic)
3. auto int y=20;//automatic variable
4. }
```

External Variable

We can share a variable in multiple C source files by using an external variable. To declare an external variable, you need to use extern keyword.

```
1. extern int x=10;//external variable (also global)
```

program1.c

```
1. #include "myfile.h"
2. #include <stdio.h>
3. void printValue(){
4.     printf("Global variable: %d", global_variable);
5. }
```

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -04 (UNIT-02)

C OPERATORS

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise, etc.

There are following types of operators to perform different types of operations in C language.

- Arithmetic Operators
- Relational Operators
- Shift Operators
- Logical Operators
- Bitwise Operators
- Ternary or Conditional Operators
- Assignment Operator
- Misc Operator

Precedence of Operators in C

The precedence of operator species that which operator will be evaluated first and next. The associativity specifies the operator direction to be evaluated; it may be left to right or right to left.

Let's understand the precedence by the example given below:

1. int value=10+20*10;

The value variable will contain 210 because * (multiplicative operator) is evaluated before + (additive operator).

The precedence and associativity of C operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ - -	Left to right
Unary	+ - ! ~ ++ - - (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right

Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR	|	Left to right
Logical AND	&&	Left to right
Logical OR	||	Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %=>>= <<= &= ^= |=	Right to left
Comma	,	Left to right

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -05 (UNIT-02)

Formatted I/O Functions

Formatted I/O functions are used to take various inputs from the user and display multiple outputs to the user. These types of I/O functions can help to display the output to the user in different formats using the format specifiers. These I/O supports all data types like int, float, char, and many more.

Why they are called formatted I/O?

These functions are called formatted I/O functions because we can use format specifiers in these functions and hence, we can format these functions according to our needs.

LIST OF SOME FORMAT SPECIFIERS-

S NO.	Format Specifier	Type	Description
1	%d	int/signed int	used for I/O signed integer value
2	%c	Char	Used for I/O character value
3	%f	Float	Used for I/O decimal floating-point value
4	%s	string	Used for I/O string/group of characters
5	%ld	long int	Used for I/O long signed integer value
6	%u	unsigned int	Used for I/O unsigned integer value
7	%i	unsigned int	used for the I/O integer value
8	%lf	double	Used for I/O fractional or floating data
9	%n	prints	prints nothing

The following formatted I/O functions will be discussed in this section-

1. printf()
2. scanf()
3. sprintf()
4. sscanf()

printf():

printf() function is used in a C program to display any value like float, integer, character, string, etc on the console screen. It is a pre-defined function that is already declared in the stdio.h(header file).

Syntax 1:

To display any variable value.

printf("Format Specifier", var1, var2, …., varn);

Example:

```
// C program to implement
// printf() function
#include <stdio.h>

// Driver code
int main()
{
    // Declaring an int type variable
    int a;

    // Assigning a value in a variable
    a = 20;

    // Printing the value of a variable
    printf("%d", a);

    return 0;
}
```

Output

20

Syntax 2:

To display any string or a message

printf("Enter the text which you want to display");

Example:

```
// C program to implement
// printf() function
#include <stdio.h>

// Driver code
int main()
{
    // Displays the string written
    // inside the double quotes
    printf("This is a string");
    return 0;
}
```

Output

```
This is a string
```

scanf():

scanf() function is used in the C program for reading or taking any value from the keyboard by the user, these values can be of any data type like integer, float, character, string, and many more. This function is declared in stdio.h(header file), that's why it is also a pre-defined function. In scanf() function we use &(address-of operator) which is used to store the variable value on the memory location of that variable.

Syntax:

scanf("Format Specifier", &var1, &var2, …., &varn);

Example:

```
// C program to implement
// scanf() function
#include <stdio.h>
```

```
// Driver code
int main()
{
    int num1;

    // Printing a message on
    // the output screen
    printf("Enter a integer number: ");

    // Taking an integer value
    // from keyboard
    scanf("%d", &num1);

    // Displaying the entered value
    printf("You have entered %d", num1);

    return 0;
}
```

Output

```
Enter a integer number: You have entered 0
```

Output:

```
Enter a integer number: 56
You have entered 56
```

sprintf():

sprintf stands for “string print”. This function is similar to printf() function but this function prints the string into a character array instead of printing it on the console screen.

Syntax:

sprintf(array_name, “format specifier”, variable_name);

Example:

```
// C program to implement
// the sprintf() function
#include <stdio.h>
  // Driver code
int main()
{    char str[50];
   int a = 2, b = 8;
     // The string "2 and 8 are even number"
   // is now stored into str
   sprintf(str, "%d and %d are even number",
           a, b);
     // Displays the string
   printf("%s", str);
   return 0;
}
```

Output

2 and 8 are even number

sscanf():

sscanf stands for “string scanf”. This function is similar to scanf() function but this function reads data from the string or character array instead of the console screen.

Syntax:

sscanf(array_name, “format specifier”, &variable_name);

Example:

```
// C program to implement
// sscanf() function
#include <stdio.h>
  // Driver code
int main()
{
    char str[50];
    int a = 2, b = 8, c, d;
      // The string "a = 2 and b = 8"
    // is now stored into str
    // character array
    sprintf(str, "a = %d and b = %d",
            a, b);
      // The value of a and b is now in
    // c and d
    sscanf(str, "a = %d and b = %d",
           &c, &d);
      // Displays the value of c and d
    printf("c = %d and d = %d", c, d);
    return 0;
}
```

Output

c = 2 and d = 8

Unformatted Input/Output functions

Unformatted I/O functions are used only for character data type or character array/string and cannot be used for any other datatype. These functions are used to read single input from the user at the console and it allows to display the value at the console.

Why they are called unformatted I/O?

These functions are called unformatted I/O functions because we cannot use format specifiers in these functions and hence, cannot format these functions according to our needs.

The following unformatted I/O functions will be discussed in this section-

1. getch()
2. getche()
3. getchar()
4. putchar()
5. gets()
6. puts()
7. putch()

getch():

getch() function reads a single character from the keyboard by the user but doesn't display that character on the console screen and immediately returned without pressing enter key. This function is declared in conio.h(header file). getch() is also used for hold the screen.

Syntax:

getch();

or

variable-name = getch();

Example:

```
// C program to implement

// getch() function

#include <conio.h>

#include <stdio.h>

  // Driver code

int main()

{

   printf("Enter any character: ");

     // Reads a character but
```

```
   // not displays
   getch();
     return 0;
}
```

Output:

Enter any character:

getche():

getche() function reads a single character from the keyboard by the user and displays it on the console screen and immediately returns without pressing the enter key. This function is declared in conio.h(header file).

Syntax:

getche();

or

variable_name = getche();

Example:

```
// C program to implement
// the getche() function
#include <conio.h>
#include <stdio.h>
  // Driver code
int main()
{
   printf("Enter any character: ");
     // Reads a character and
   // displays immediately
```

```
    getche();
    return 0;
}
```

Output:

Enter any character: g

getchar():

The getchar() function is used to read only a first single character from the keyboard whether multiple characters is typed by the user and this function reads one character at one time until and unless the enter key is pressed. This function is declared in stdio.h(header file)

Syntax:

Variable-name = getchar();

Example:

```
// C program to implement
// the getchar() function
#include <conio.h>
#include <stdio.h>
  // Driver code
int main()
{
    // Declaring a char type variable
    char ch;
      printf("Enter the character: ");
      // Taking a character from keyboard
    ch = getchar();
    // Displays the value of ch
    printf("%c", ch);
```

```
    return 0;
}
```

Output:

Enter the character: a

a

putchar():

The putchar() function is used to display a single character at a time by passing that character directly to it or by passing a variable that has already stored a character. This function is declared in stdio.h(header file)

Syntax:

putchar(variable_name);

Example:

```
// C program to implement
// the putchar() function
#include <conio.h>
#include <stdio.h>
 // Driver code
int main()
{
    char ch;
    printf("Enter any character: ");
     // Reads a character
    ch = getchar();
     // Displays that character
    putchar(ch);
    return 0;
```

```
}
```

output:

Enter any character: Z

Z

gets():

gets() function reads a group of characters or strings from the keyboard by the user and these characters get stored in a character array. This function allows us to write space-separated texts or strings. This function is declared in stdio.h(header file).

Syntax:

char str[length of string in number]; //Declare a char type variable of any length

gets(str);

Example:

```
// C program to implement
// the gets() function
#include <conio.h>
#include <stdio.h>
  // Driver code
int main()
{
    // Declaring a char type array
    // of length 50 characters
    char name[50];
      printf("Please enter some texts: ");
      // Reading a line of character or
    // a string
```

```
    gets(name);
     // Displaying this line of character
    // or a string
    printf("You have entered: %s",
          name);
    return 0;
}
```

Output:

Please enter some texts: geeks for geeks

You have entered: geeks for geeks

puts():

In C programming puts() function is used to display a group of characters or strings which is already stored in a character array. This function is declared in stdio.h(header file).

Syntax: puts(identifier_name);

Example:

```
// C program to implement
// the puts() function
#include <stdio.h>
// Driver code
int main()
{
    char name[50];
    printf("Enter your text: ");
    // Reads string from user
    gets(name);
```

```
    printf("Your text is: ");
    // Displays string
   puts(name);
     return 0;}
```

Output:

Enter your text: GeeksforGeeks

Your text is: GeeksforGeeks

putch():putch() function is used to display a single character which is given by the user and that character prints at the current cursor location. This function is declared in conio.h(header file)

Syntax: putch(variable_name);

Example:

```
// C program to implement
// the putch() functions
#include <conio.h>
#include <stdio.h>
  // Driver code
int main()
{char ch;
    printf("Enter any character:\n ");
      // Reads a character from the keyboard
    ch = getch();
      printf("\nEntered character is: ");
      // Displays that character on the console
    putch(ch);
    return 0;
```

}

Output:

Enter any character:

Entered character is: d

Formatted I/O vs Unformatted I/O

S No.	Formatted I/O functions	Unformatted I/O functions
1	These functions allow us to take input or display output in the user's desired format.	These functions do not allow to take input or display output in user desired format.
2	These functions support format specifiers.	These functions do not support format specifiers.
3	These are used for storing data more user friendly	These functions are not more user-friendly.
4	Here, we can use all data types.	Here, we can use only character and string data types.
5	printf(), scanf, sprintf() and sscanf() are examples of these functions.	getch(), getche(), gets() and puts(), are some examples of these functions.

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.

LECTURE -06 (UNIT-02)

C TYPE CONVERSION

In this tutorial, you'll learn about type conversion in C programming with the help of examples.

In C programming, we can convert the value of one data type (int, float, double, etc.) to another. This process is known as type conversion. Let's see an example,

```
#include <stdio.h>

int main() {

  int number = 34.78;

  printf("%d", number);

  return 0;
}

// Output: 34
```

Run Code

Here, we are assigning the double value 34.78 to the integer variable number. In this case, the double value is automatically converted to integer value 34.

This type of conversion is known as implicit type conversion. In C, there are two types of type conversion:

1. Implicit Conversion
2. Explicit Conversion

Implicit Type Conversion In C

As mentioned earlier, in implicit type conversion, the value of one type is automatically converted to the value of another type. For example,

```
#include<stdio.h>

int main() {
```

```
  // create a double variable
  double value = 4150.12;
  printf("Double Value: %.2lf\n", value);

  // convert double value to integer
  int number = value;

  printf("Integer Value: %d", number);

  return 0;
}
```

Run Code

Output

```
Double Value: 4150.12
Integer Value: 4150
```

The above example has a double variable with a value 4150.12. Notice that we have assigned the double value to an integer variable.

```
int number = value;
```

Here, the C compiler automatically converts the double value 4150.12 to integer value 4150.

Since the conversion is happening automatically, this type of conversion is called implicit type conversion.

Example: Implicit Type Conversion

```
#include<stdio.h>

int main() {

  // character variable
  char alphabet = 'a';
  printf("Character Value: %c\n", alphabet);

  // assign character value to integer variable
  int number = alphabet;

  printf("Integer Value: %d", number);
```

```
  return 0;
}
```
Run Code

Output

```
Character Value: a
Integer Value: 97
```

The code above has created a character variable *alphabet* with the value 'a'. Notice that we are assigning *alphabet* to an integer variable.

```
int number = alphabet;
```

Here, the C compiler automatically converts the character 'a' to integer 97. This is because, in C programming, characters are internally stored as integer values known as ASCII Values.

ASCII defines a set of characters for encoding text in computers. In ASCII code, the character 'a' has integer value 97, that's why the character 'a' is automatically converted to integer 97.

If you want to learn more about finding ASCII values, visit find ASCII value of characters in C.

Explicit Type Conversion In C

In explicit type conversion, we manually convert values of one data type to another type. For example,

```
#include<stdio.h>

int main() {

  // create an integer variable
  int number = 35;
  printf("Integer Value: %d\n", number);

  // explicit type conversion
  double value = (double) number;

  printf("Double Value: %.2lf", value);

  return 0;
}
```

Output

```
Integer Value: 35
Double Value: 35.00
```

We have created an integer variable named *number* with the value 35 in the above program. Notice the code,

```
// explicit type conversion
double value = (double) number;
```

Here,

- (double) - represents the data type to which *number* is to be converted
- number - value that is to be converted to double type

Example: Explicit Type Conversion

```
#include<stdio.h>

int main() {

  // create an integer variable
  int number = 97;
  printf("Integer Value: %d\n", number);

  // (char) converts number to character
  char alphabet = (char) number;

  printf("Character Value: %c", alphabet);

  return 0;
}
```

Run Code

Output

```
Integer Value: 97
Character Value: a
```

We have created a variable number with the value 97 in the code above. Notice that we are converting this integer to character.

char alphabet = (char) number;

Here,

- (char) - explicitly converts *number* into character
- number - value that is to be converted to char type

Data Loss In Type Conversion

In our earlier examples, when we converted a double type value to an integer type, the data after decimal was lost.

```
#include<stdio.h>

int main() {

  // create a double variable
  double value = 4150.12;
  printf("Double Value: %.2lf\n", value);

  // convert double value to integer
  int number = value;

  printf("Integer Value: %d", number);

  return 0;
}
```

Run Code

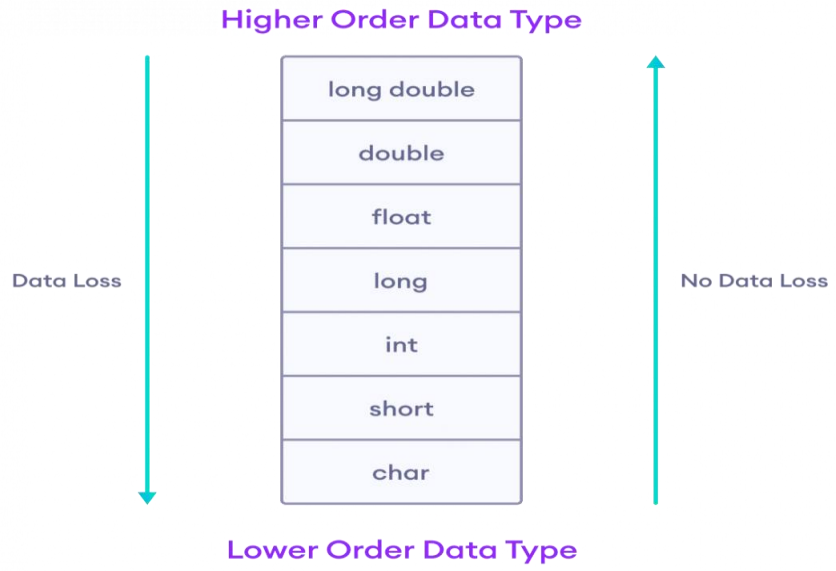
Output

```
Double Value: 4150.12
Integer Value: 4150
```

Here, the data 4150.12 is converted to 4150. In this conversion, data after the decimal, .12 is lost.

This is because double is a larger data type (8 bytes) than int (4 bytes), and when we convert data from larger type to smaller, there will be data loss..

Similarly, there is a hierarchy of data types in C programming. Based on the hierarchy, if a higher data type is converted to lower type, data is lost, and if lower data type is converted to higher type, no data is lost.



Possible Data Loss During C Type Conversion

Here,

- data loss - if long double type is converted to double type
- no data loss - if char is converted to int

What is Mixed Mode Arithmetic Expression?

In a statement or expression if one the operand is real (float) and another one is integer then expression is called as Mixed Mode Arithmetic Expression. If in an expression either operand is of real then output is always in real format. If both operands are real then output will be in real formats.

Example

Real operand and Integer operand = Real operand (Output)
Real operand and Real operand = Real operand (Output).

Types of Arithmetic Operators used in Mixed Mode Expression

"*" - Multiply use for Multiplication
"/" - Divide use for division
"+" - Plus use for addition
"-" - Minus use for subtraction
"%" - Modulus use for getting a reminder
"**" - Square use for getting square number

Rules for Evaluation Mixed Mode Arithmetic Expression

Rule 1

Evaluate Expressions always from Left to Right

For Example: 3 + 5 - 4 = 4

Rule 2

Priority of an operator is also considered while calculating an expression

Operator Priorities in descending order

1. (**) Known as Square operator which executes From Right To Left .

For example 5**2 is equal to 25

2. (*) Known as multiplication operator which executes From Left to Right

For example 5 * 2 is equal to 10

3. (/) Known as Division operator which executes From Left to Right

For example 6 / 2 is equal to 3

4. (+) Known as Plus or Addition operator which executes From Left to Right

For example 6 + 2 is equal to 8

5. (-) Known as Minus or Subtraction operator which executes From Left to Right

For example 6 - 2 is equal to 4

Mixed mode operator example

2 + 2 * 5 ** 3

==> 2 + 2 * (5**3)
==> 2 + 2 * 125
==> 2 + (2 * 125)
==> 2 + 250 = 252

Rule 3

If the operands of this operator are of the same type means either Integer or Real, compute the same for result in that operator only.

For Example

Both Real Numbers

double a = 2.0;
double b = 3.0;

Double add = 2.0 + 3.0 = 5.0 (Result in Real Number only)

Both Integer Number

int a = 2;
int b = 3;
int add = 2 + 3 = 5 (Result in Integer Number only)

Rule 4

If one of the operator is Integer and other one is real then convert integer number to real number and compute the result in real number only.

For Example

double a = 5.0;

int b = 5;

Result = 5.0 = 5 = 10.0 (Result in Real Number only).

REFERENCES:-

1. Brian W.Kernighan and Dennis Ritchie,C Programming Langauge,Prantice Hall of India.
2. Yashwant Kanetkar,Let US C,BPB Publication.